Local LLM Coding Agent for SWE-Bench Verified

Mohammadreza Zia

1. Aim and setup

I wanted to see how far a coding agent could get while running entirely on my own machine. I used **Granite 4.2 8B** through **Ollama** (temperature 0, 8192-token context) on a Lenovo Legion 5 with an Intel Core i7-10750H, a GTX 1660 Ti with 6 GB VRAM, and 8 GB RAM. Larger local models were too slow or too demanding for repeated experiments, so I put my effort into the agent around the model instead.

The exercise suggests Claude Code or an open-source alternative. I first tried OpenWork/OpenCode and mini-SWE-agent, but with small local models they gave tool-format errors, repeated actions and loops. I therefore moved from the suggested tools to a small custom Python agent that used the same local Granite model.

I selected ten SWE-Bench Verified issues from Django, scikit-learn, SymPy, Sphinx, pytest and xarray, favouring relatively small and localised repairs that were practical for the available hardware. I inspected the reference patches offline only for task selection; they were never provided to the agent. The agent itself receives only the issue statement and the repository at the base commit, with no hints about files or root causes.

2. Agent design and development

The agent has four parts:

- **Retrieval.** Using only the issue text, it extracts identifiers (CamelCase, snake_case, backticked names) and scores files by IDF-weighted occurrences, with bonuses for exact class/function definitions and imports. Source and test files are ranked in separate pools so frequent matches in tests do not crowd out source files.
- **Bounded context.** Files are shown as excerpts around matches (at most 9000 characters), including line ranges mentioned in the issue and, in the final version, focused source context built from traceback file and line references.
- **Two-stage actions.** The model first chooses `read`, `inspect`, `search` or `edit`, and then fills an action-specific JSON schema (for example, `read` only accepts unread files).
- **Validated edits.** Edits use an `old_text/new_text` format. Before applying, the script checks that `old_text` occurs exactly once in the selected file, the result is valid Python, the change is not comment-only or an obvious no-op, and no test file is edited. Rejections are returned to the model with the reason.

I did not design this up front. I added features one at a time after failures, so every issue also served as development data. In one SymPy task the agent inspected the same function repeatedly, so I added a duplicate-inspection check and repository search. A Sphinx task showed that the model could pick the right action but return the wrong fields, which led to the two-stage protocol. The biggest improvement came from xarray: an earlier version had too much context and drifted, and traceback-based context kept the edit stage on `coerce_pandas_values()`, where the agent correctly saw that `DataArray` needed special handling.

3. Results

The final agent version (SHA-256 5589ed3415e3) was frozen and evaluated across all ten tasks. The table reports the frozen-version results rather than selecting the strongest result from earlier development runs. **The official SWE-Bench harness was not run**, mainly because of limited disk space. "Pass" therefore means the patch survived my own local checks below, not that the issue is officially resolved, and the hidden FAIL_TO_PASS tests were not applied. The agent produced **3 locally validated repair candidates out of 10**.

Task	Local check	Main outcome
Django 10880	Fail	Introduced a `COUNT(*)` spacing regression
scikit-learn 14141	Pass	Added `joblib` to dependency reporting
Django 11179	Fail	Generated a duplicate delete operation
Django 16082	Fail	Found the right area but produced an incomplete MOD fix
SymPy 16886	Fail	Understood the bug but got stuck generating malformed edits
SymPy 15809	Fail	Got stuck in an inspect/edit recovery loop
SymPy 13757	Fail	Applied a valid patch, but behaviour was still wrong
Sphinx 8269	Fail	Drifted into unrelated code
pytest 7982	Pass	Restored symlink traversal correctly
xarray 3677	Pass	Correctly handled merging a `DataArray` into a `Dataset`

Validation. For pytest (run under Python 3.8), the relevant symlink tests gave 4 passed, 75 deselected. For xarray, `Dataset.merge(DataArray)` matched `xr.merge([Dataset, DataArray])` (`identical = True`) and the relevant merge tests gave 24 passed, 870 deselected. The scikit-learn checkout could not run its test module because the compiled extension was not built, so I loaded the modified file directly and checked that `_get_deps_info()` returned `joblib`. Validation was therefore uneven across tasks.

Some earlier versions did better on individual tasks than the final one. The SymPy Morse-code issue was solved earlier with a tiny edit; in the final version the model still knew the exact change, but the stricter edit process kept producing malformed replacement text. An earlier version also passed the relevant Sphinx tests, while the final version drifted into unrelated LaTeX code. Per-task logs, patches where applicable, validation summaries, and available test or functional-validation output are in `results/`, and selected earlier runs are in `development_history/`.

4. Analysis and limitations

Finding the code was easier than repairing it. Several failed runs reached the right file and function, but the patch was incomplete or wrong. The model often seemed to understand the bug, and problems appeared when that understanding had to become an exact edit: repeated actions, too much context, malformed replacement text and poor recovery all caused failures.

More control was not always better. Focused, traceback-based context clearly helped a harder task such as xarray, but stricter edit handling sometimes made simpler tasks harder, as in the SymPy Morse-code case. The final agent was not uniformly better than earlier ones; it traded some failure modes for others.

Validation mattered. The Django aggregation patch looked plausible until checking showed it turned `COUNT(*)` into `COUNT( *)`, and the SymPy multiplication patch was valid Python but did not fix the behaviour.

Limitations. All ten issues were also used to develop the agent, so 3/10 is a development-set result, not a held-out estimate, and with ten issues it is very noisy. While debugging failed runs I sometimes analysed an issue's root cause myself; none of this was given to the agent, but it may have influenced which generic features I added. The official harness was not run. With 6 GB VRAM and 8 GB RAM I could not fairly test the same agent with a much larger local model, which may have produced different reasoning and edit-generation behaviour. Although temperature was set to 0, local model runs are not guaranteed to be perfectly reproducible.

AI assistance

I used ChatGPT and Claude (Anthropic) as development assistants to help draft code, shell commands, experiment notes and this report. I decided what the agent should do, ran all experiments myself, reviewed the outputs, and checked the patches and test results. The SWE-Bench repairs themselves were produced only by the locally deployed Granite model through Ollama. No GPT, Claude or other cloud-hosted model generated any repair patch.